

Fuel Route

Project Description

(Lexon, Samuel, Dalton, Arielle, Jelon)

Problem / solution:

The existing gas station and navigation applications may help to find the nearby gas stations and fuel prices but they do not always combine this information with the fuel requirements of a specific car. The drivers have to separately determine what fuel is required for their car and then compare the nearest gas stations based on prices and distance.

Fuel Route is motivated by combining all these steps into one application. The user chooses their vehicle and location, and the system determines the most compatible type of gas, loads the nearby gas stations and available prices, calculates the distance and ranks the results. To support the distributed-computing goals, the station data is divided into independent batches and processed concurrently by multiple worker services before being combined into a final ranked response.

Core workflow:

Location/GPS → vehicle selection → determine fuel requirement → retrieve nearby stations/prices → process results in parallel → rank/filter → show on map/list.

Core features:

- GPS/manual location
- Vehicle lookup
- Fuel compatibility
- Nearby station search
- Gas prices
- Distance calculation
- Map/list results
- Basic ranking of best gas stations.

Distributed-computing idea:

Split nearby station/data processing among multiple workers, then combine the results into the final ranked list. We should define the exact worker responsibilities later, but this needs to be a real part of the system rather than just a label.

Scope:

Build a functional prototype first; things like accounts, favorites, notifications, historical prices, crowdsourced prices, etc. are optional/stretch features.

Milestones:

Basic app + location/vehicle → station/price data → distributed processing/ranking → integration/testing → final demo.

Final deliverable:

A working application where a user can select a vehicle and location and receive nearby gas-station recommendations with compatible fuel, price, and distance.

Technical Stack

(Platform: Web)

Tool, Package, Language, Etc.	What is it for?
Typescript	Primary programming language for web application, backend APIs, coordinator, and worker services
Next.js	Full-stack web framework for frontend, routing, and server-side API endpoints
Mantine	React component library for building UI
Leaflet	Interactive map, markers, popups, and other map functionality
OpenStreetMap	Map tiles and geographic data used to locate nearby gas stations
Overpass API	Queries OpenStreetMap data to retrieve gas stations near a latitude/longitude
Supabase / PostgreSQL	Stores crowdsourced gas prices, station references, and other persistent application data
AWS EC2	Hosts distributed coordinator/worker services used to process station batches concurrently
Vercel	Hosts/deploys the Next.js web application
Vitest	Unit and integration testing
FuelEconomy.gov	Vehicle information and fuel-related data used to determine appropriate fuel for selected vehicles

Member Breakdown

Key:

- **Green**: Mostly Responsible (Primary Lead)
- **Orange**: Semi-Responsible (Help Assist the Lead)
- **Red**: Not Responsible

	Lexon	Samuel	Dalton	Arielle	Jelon
API/Data Research	Red	Orange	Orange	Red	Green
UI/UX	Red	Red	Green	Orange	Green
Location/GPS	Green	Orange	Orange	Orange	Orange
Openstreet + Leaflet + Overpass API	Green	Red	Red	Red	Orange
Vehicle Lookup	Orange	Red	Green	Orange	Orange
Fuel Compatibility	Green	Orange	Green	Red	Orange
Gas Price Data	Red	Orange	Orange	Orange	Green
Gas Station Search	Green	Red	Orange	Green	Orange
Distance calculation	Red	Orange	Red	Green	Red
Ranking Algorithm	Red	Green	Red	Red	Red
Worker Architecture + Implementation	Orange	Green	Orange	Red	Orange
Testing	Orange	Orange	Green	Orange	Orange
AWS/Deployment	Green	Green	Green	Green	Green
Documentation	Orange	Orange	Orange	Orange	Orange

Phase Breakdown

Phase 0 - Project Setup

Goal: Everyone can develop locally and contribute without the project breaking.

- GitHub Repository Setup + Sharing: (Lexon – Primary / Dalton – Support – Finish by Oct. 3rd)
 - Main vs Beta branch.
 - Main branch protection rules - Require another person's approval before pushing to main.
 - Commit and PR titles and descriptions conventions.
 - ReadMe file updated to match this project document.
 - "Conventions" file for GitHub and Codebase conventions.
- Create Next.js project: (Lexon – Primary / Dalton – Support – Finish by Oct. 3rd)
- Install/configure Mantine: (Lexon – Primary / Dalton – Support – Finish by Oct. 3rd)
- API/Data research: (Jelon – Primary / Samuel & Dalton – Support – Finish by Oct. 3rd)
 - [FuelEconomy.gov](https://fuel-economy.gov)
 - Research available free/public gas-price sources.
 - Determine whether live station-level pricing is feasible.
 - OpenStreetMap/Leaflet

Phase 1 - Basic UI, Location, & Vehicle

Goal: Users are able to open Fuel Route to provide a location and selected vehicle.

- Basic site layout/navigation: (Dalton & Jelon – Primary / Arielle – Support – Finish by Oct. 10th)
- Home/search page: (Dalton & Jelon – Primary / Arielle – Support – Finish by Oct. 10th)
- GPS location (Lexon – Primary / Arielle – Support – Finish by Oct. 3rd)
- Manual location entry (Lexon – Primary / Arielle – Support – Finish by Oct. 3rd)
- Vehicle selection: (Dalton – Primary / Lexon, Arielle & Jelon – Support – Finish by Oct. 10th)
 - Year, Make, Model, Trim/Engine, Etc
- Determine required/recommended fuel types for selected vehicle (Jelon - Finish by Oct. 10th)
- Basic error/loading states: (Dalton & Jelon – Primary / Arielle – Support – Finish by Oct. 17th)

Phase 2 - Stations, Prices, & Map

Goal: Retrieve actual nearby stations and show useful information to users.

- Query nearby gas stations (Lexon & Arielle – Primary / Dalton & Jelon – Support – Finish by Oct. 17th)
- Integrate Leaflet map with OpenStreetMap tiles and display the user's location and nearby gas stations on the map using Overpass API (Lexon – Primary / Jelon – Support – Finish by Oct. 17th)
 - Set up Leaflet in the Next.js application.
 - Load OpenStreetMap as the map tile provider.
 - Center the map on the user's GPS/manual location.
 - Set an appropriate default zoom level.
 - Display the user's location on the map.
 - Query nearby gas stations using the Overpass API.
 - Parse station coordinates and information returned by Overpass.
 - Display nearby gas stations as map markers.
 - Display basic station information when a marker is selected.
- Determine gas prices for different fuel types: (Jelon – Primary / Samuel, Dalton & Arielle – Support – Finish by Oct. 24th)
- Calculate distance from user's location: (Arielle – Primary / Samuel – Support – Finish by Oct. 24th)

Phase 3 - Distributed Processing

Goal: Implement a distributed station-processing system using a coordinator service and multiple independent worker services hosted on AWS EC2. The coordinator will divide station data into batches, distribute the batches to workers using HTTP/JSON requests, and aggregate the returned results into a final response.

- Create coordinator/distributor service. (Samuel – Primary / Lexon – Support – Finish by Oct. 31st)
 - Host coordinator service on an AWS EC2 instance.
 - Receive user location, required fuel type, and nearby station data.
 - Generate a unique ID for each processing job.
 - Determine available workers
- Divide station data into batches. (Samuel – Primary / Lexon & Jelon – Support – Finish by Oct. 31st)
 - Split stations approximately evenly based on the number of available workers.
 - Associate each batch with the processing job and assigned worker.
- Dispatch batches to multiple worker processes/services concurrently. (Samuel – Primary / Jelon – Support – Finish by Nov. 7th)

- Each worker: (Samuel – Primary / Jelon – Support – Finish by Nov. 7th)
 - Each worker independently processes its assigned stations.
 - Associate available crowdsourced gas-price data with each station.
 - Determine fuel compatibility.
 - Calculate distance from the user.
 - Calculate values needed for station ranking.
 - Return processed station results to the coordinator as JSON.
- Return worker results to coordinator. (Samuel – Primary / Dalton – Support – Finish by Nov. 7th)
- Aggregate worker responses into one station collection. (Samuel – Primary / Lexon – Support – Finish by Nov. 14th)
 - Coordinator waits for worker responses.
 - Match returned results using job/station IDs.
 - Combine worker results into one station collection.
 - Prevent duplicate station results.
 - Filter invalid or incompatible results.
 - Perform final ranking/sorting.
 - Return the completed station results to the web application.
- Handle failed/timed-out workers. (Samuel – Primary / Dalton & Jelon – Support – Finish by Nov. 14th)
 - Set timeouts for worker requests.
 - Detect failed or unavailable workers.
 - Reassign failed batches to another available worker.
 - Log worker failures and processing times.
 - Ensure one failed worker does not cause the entire processing request to fail when another worker is available.
- Measure processing time. (Samuel – Primary / Dalton – Support – Finish by Nov. 21st)
- Compare single-worker vs multi-worker execution. (Samuel – Primary / Dalton & Jelon – Support – Finish by Nov. 21st)

Phase 4 - Ranking & Recommendation

Goal: Turn raw station data into useful recommendations.

- Define ranking algorithm (Samuel – Primary – Finish by Nov. 7th)
- Filter incompatible fuel. (Samuel – Primary / Lexon & Dalton – Support – Finish by Nov. 14th)
- Rank by fuel price, distance, possibly combined score. (Samuel – Primary – Finish by Nov. 14th)
- User sorting for cheapest, closest, best overall. (Samuel – Primary / Dalton & Jelon – UI Support – Finish by Nov. 14th)

Phase 5 - Testing, Deployment, & Demo

Goal: Integrate all components, deploy the system/app, and prepare the app for final demo.

- Combine all branches/components from everyone on the team (All Members - Finish by Nov. 21st)
- Unit testing (All Members - Finish by Nov. 21st)
- Test entire app: GPS, Maps, Gas prices, Fuel compatibility, etc. (All Members - Finish by Nov. 21st)
- Test Distributed system: Multiple workers processing batches concurrently. (All Members - Finish by Nov. 21st)
- Deploy Next.js web app to Vercel (All Members - Finish by Nov. 21st)
 - Connect prod GitHub to Vercel
 - Configure production environment variables
 - Verify production build/deployment
- Deploy distributed services to AWS EC2 (All Members - Finish by Nov. 21st)
 - Create/configure AWS EC2 instances
 - Deploy coordinator and worker services
 - Install Node.js and required project dependencies
 - Configure environment variables and Supabase/API credentials
 - Configure networking for worker communication
 - Start the coordinator and distributed worker services
 - Verify the deployed application can communicate with the workers.
- Update documents such as ReadMe, Architecture diagram, Tech Stack, etc. (All Members - Finish by Nov. 21st)
- Demo the app. (All Members - Finish by Nov. 21st)